

Why This Problem Matters

Grid problems show up constantly in coding interviews because they compress a lot of algorithmic thinking into a small, visual space. LeetCode 3568, Minimum Moves to Clean the Classroom, looks like a simple pathfinding exercise at first glance, but it's actually testing something sharper: can you recognize when a resource constraint (energy) needs to become part of your search *state* rather than part of your search *cost*?

Interviewers love this distinction because it's a common trap. The instinct when you see "energy" or "fuel" in a problem is to reach for Dijkstra's algorithm, since it feels like a weighted-graph problem. But every move here costs exactly one step — the energy is a *constraint* on which moves are legal, not a *weight* on the edges. Spotting that difference, and then figuring out how to track energy and collected litter without blowing up your state space, is exactly the kind of reasoning that separates a candidate who memorized BFS templates from one who understands why BFS works.

Problem Overview

You're given a grid representing a classroom. Each cell is one of:

- **S** — the student's starting position
- **L** — a litter cell that needs to be collected
- **R** — a reset cell that fully restores the student's energy the instant they step on it
- **X** — an obstacle the student cannot walk through
- An empty floor cell

The student starts with a fixed maximum energy. Every move to an adjacent cell (up, down, left, right) costs exactly one unit of energy. If the student steps onto a reset cell, their energy is restored to full immediately, regardless of what it was before. The student needs to visit every litter cell at least once, in as few total moves as possible. If it's impossible to collect all the litter — because energy runs out before reaching some cell and no reset can save it — the answer is `-1`.

Example

Consider a small 2×2 grid:

L S
R L

Energy cap: 4.

- Start on **S**, move left onto **L** → litter collected, energy now 3.
- Move down onto **R** → energy resets to full (4), even though it had only dropped to 2.
- Move right onto **L** → second litter collected.

Total: **3 moves**. The reset is what makes this route efficient — without it, a longer detour might be needed to avoid running dry.

Intuition

The first thing to notice is what stays constant and what varies. Every move costs exactly one step, no matter what cell you land on. That constancy is the signal that BFS — not Dijkstra — is the right tool, since BFS guarantees the shortest path in an unweighted graph, and this graph is unweighted in terms of move cost.

What varies is *whether a move is even legal*. You can't take a step if you have zero energy left. And whether litter is "collected" depends on your path history — you need to remember which of it you've already picked up.

So the real question becomes: what is a "node" in this graph? It's not just a grid cell. Two students standing on the same cell can be in completely different situations — one might have full energy and all litter collected, another might be almost out of energy with half the litter still dirty. Those are different states, even though they're on the same square. The state has to capture everything that affects future moves: **position, remaining energy, and which litter has been collected**.

The litter constraint gives you the final piece: at most 10 litter cells. Ten yes/no flags is exactly what a bitmask is for — pack the collected/uncollected status of every litter cell into a single integer instead of a list or set. Now an entire state fits into four small numbers, and comparing two states for equality is instant.

Brute Force Solution

Idea: Simulate every possible route from the start, branching in four directions at every step, and track which litter has been picked up along each path.

Algorithm: At each cell, recursively try all four neighbors, updating a running set of collected litter, until either all litter is collected (record the move count) or energy runs out (abandon the path).

Advantages: Conceptually simple, no special data structures needed.

Disadvantages: Reset cells let the student revisit cells they've already visited, meaning a path can loop indefinitely. There's no natural stopping condition, so raw path enumeration doesn't even converge to a fixed number of routes — let alone a manageable one. On a 20×20 grid this isn't just slow, it's structurally broken.

Time complexity: Unbounded / exponential (does not terminate cleanly due to loops). **Space complexity:** Unbounded for the same reason.

Optimal Solution

The fix is to stop thinking about *paths* and start thinking about *states*. A state is:

```
(row, col, energy, litter_mask)
```

If the search ever revisits an identical state, there's no point exploring it again — some other path already reached that exact combination of position, energy, and progress in the same number of moves or fewer (since BFS explores in non-decreasing move order). This collapses an infinite tree of paths into a finite graph of states.

Grid dimension	Max value	Why it matters
Rows × Cols	400	Bounds position
Energy	50	Bounds the third state field
Litter cells	10	Bounds mask to $2^{10} = 1024$ values

Total state space: $\text{rows} \times \text{cols} \times \text{energy} \times 2^{\text{litter_count}}$ — roughly 20 million in the worst case, which BFS chews through easily.

BFS explores states layer by layer: all states reachable in 1 move, then all states reachable in 2 moves, and so on. Because it never skips a layer, the very first time a state has every litter bit cleared, that layer number is the minimum move count — no other path could possibly have reached "fully clean" in fewer moves, since BFS would have found it first.

The rule that actually encodes the problem: when you move onto a cell, - if it's a reset cell, new energy = max energy, no matter what it was before - otherwise, new energy = current energy - 1 - if it's an uncollected litter cell, flip that bit off in a *fresh copy* of the mask

The one gotcha: the mask check for "all litter collected" must happen the instant you generate a new state, *before* you check or add it to the visited set. That's what lets you return immediately on the very move that finishes the job, rather than doing one extra unnecessary loop.

Python Code

```
from collections import deque

def min_moves_to_clean(grid: list[list[str]], energy_cap: int) -> int:
    rows, cols = len(grid), len(grid[0])

    start = None
    litter_index = {}
    for r in range(rows):
        for c in range(cols):
            cell = grid[r][c]
            if cell == 'S':
                start = (r, c)
            elif cell == 'L':
                litter_index[(r, c)] = len(litter_index)

    full_mask = (1 << len(litter_index)) - 1
    if full_mask == 0:
        return 0 # no litter to collect

    visited = {(start[0], start[1], energy_cap, 0)}
    queue = deque([(start[0], start[1], energy_cap, 0)])
    moves = 0

    while queue:
        moves += 1
        for _ in range(len(queue)):
            r, c, energy, mask = queue.popleft()
            if energy == 0:
                continue

            for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                nr, nc = r + dr, c + dc
                if not (0 <= nr < rows and 0 <= nc < cols):
                    continue
                if grid[nr][nc] == 'X':
                    continue

                new_energy = energy_cap if grid[nr][nc] == 'R' else energy - 1
                new_mask = mask
                if (nr, nc) in litter_index:
                    new_mask &= ~(1 << litter_index[(nr, nc)])
```

```

        if new_mask == full_mask:
            return moves

        state = (nr, nc, new_energy, new_mask)
        if state not in visited:
            visited.add(state)
            queue.append(state)

    return -1

```

Code Walkthrough

- **Scan the grid once** to find the start position and index every litter cell (`litter_index` maps each litter coordinate to a bit position).
- `full_mask` is the target: every litter bit turned on. If there's no litter, return 0 immediately.
- **Seed the queue** with the start state — full energy, mask 0 — and mark it visited.
- **Each while iteration is one move layer.** Draining the queue with `for _ in range(len(queue))` processes every state reachable in the current move count before advancing to the next.
- **Dead branches:** if `energy == 0`, the state has nowhere left to go, so skip it.
- **Neighbor generation:** skip off-grid cells and obstacles.
- **Energy update:** full refill on `R`, otherwise decrement by one.
- **Mask update:** flip the relevant bit off in a *new* mask value if landing on uncollected litter — never mutate the old mask, since it may still be needed by other states in the queue.
- **Early return:** the moment `new_mask == full_mask`, return the current move count without waiting to enqueue the state.
- **Visited check:** only push genuinely new states onto the queue.

Dry Run

Using the earlier 2×2 example: `L S / R L`, energy cap 4.

Move	State before	Action	State after
1	(S, energy=4, mask=00)	move left onto L	(energy=3, mask=01)
2	(energy=3, mask=01)	move down onto R	(energy=4, mask=01)
3	(energy=4, mask=01)	move right onto L	(energy=3, mask=11 = full_mask)

At move 3, `new_mask == full_mask`, so the function returns `3` immediately — matching the expected answer.

Complexity Analysis

Time: $O(\text{rows} \times \text{cols} \times \text{energy_cap} \times 2^{\text{litter_count}})$ — each unique state is enqueued and processed at most once, and generating neighbors from a state is $O(1)$ (4 directions).

Space: Same bound, for the `visited` set and queue holding up to that many states.

Plugging in worst-case limits (400 cells, energy 50, 10 litter cells) gives roughly 20 million states — large but well within what a modern machine handles in well under a second.

This bound is correct because BFS never revisits a state, and the state itself is defined precisely enough (position + energy + mask) to capture every fact that matters for future moves.

Alternative Solutions

You could implement this as **iterative deepening DFS** on the same state space, but that trades BFS's natural shortest-path guarantee for repeated re-exploration at each depth — strictly worse here since BFS already gives you the answer for free at the first layer that satisfies the goal.

A **Dijkstra-based** approach also works (every state transition still has uniform cost 1), but it's strictly more overhead — a priority queue instead of a plain queue — for no benefit, since all edge weights are equal.

Edge Cases

1. **No litter at all** — return 0 immediately, no search needed.
2. **Litter unreachable due to obstacles with no reset nearby** — the queue empties without ever reaching `full_mask`; return `-1`.
3. **Start position surrounded by obstacles** — no valid first move; queue empties on the first layer.
4. **Energy cap of 0 with litter present** — student can't move at all; return `-1` unless litter count is 0.
5. **Multiple reset cells** — algorithm handles this naturally since each reset independently sets energy to max regardless of path taken to reach it.

Common Mistakes

1. **Gating the reset on energy hitting exactly zero.** The rule is unconditional — standing on `R` always tops off energy, even with energy to spare. Gating it undercounts valid routes.
2. **Mutating the mask in place** instead of creating a fresh copy per neighbor — this corrupts sibling branches sharing the same starting mask.
3. **Checking `full_mask` after adding to visited** instead of before — this adds an unnecessary extra layer to the returned move count.
4. **Forgetting to skip states with zero energy** — leads to invalid moves being processed or infinite exploration.
5. **Using a list or set instead of a bitmask** for litter tracking — works correctness-wise, but is far slower to hash and compare, and blows up the practical state space.
6. **Treating this as a weighted-graph problem** and reaching for Dijkstra when plain BFS already guarantees correctness at lower complexity.

Interview Questions

- Why does uniform move cost make BFS sufficient instead of Dijkstra?
- How would the solution change if diagonal moves were allowed?
- What if there were more than 10 litter cells — how would you adapt the state representation?
- How would you reconstruct the actual path, not just the move count?
- What if reset cells only partially restored energy instead of fully?

Similar Problems

- **LeetCode 864: Shortest Path to Get All Keys** — nearly identical bitmask + BFS pattern, tracking collected keys instead of litter.
- **LeetCode 1293: Shortest Path in a Grid with Obstacles Elimination** — BFS with an extra state dimension (obstacles removed) instead of energy.
- **LeetCode 847: Shortest Path Visiting All Nodes** — bitmask BFS over graph nodes rather than grid cells, same core idea of packing "visited set" into an integer.

Key Takeaways

When a problem gives every move the same cost but adds a resource constraint, fold that constraint into your BFS *state*, not into edge weights. When you need to track a small set of binary flags (which items are collected, which nodes are visited), a bitmask turns an unwieldy list into a single comparable, hashable integer. And always evaluate your goal condition the moment a new state is generated —

waiting an extra cycle costs you correctness on the "first layer to satisfy the goal" guarantee that makes BFS work in the first place.

Watch the Video

Prefer to see the whole derivation and code walkthrough in motion, including the full quiz section?

Watch it here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that treats algorithm and interview problems as puzzles worth genuinely understanding — not just memorizing patterns for. Every episode goes from raw problem statement to a fully reasoned, from-scratch solution, with the "why" spelled out at every step.

Call To Action

If this walkthrough helped the state-space trick click, give the video a like on YouTube and subscribe for the rest of the series. For deeper written breakdowns delivered straight to your inbox, subscribe to the Substack. Got a different approach or a follow-up question? Drop it in the comments — and if you know someone prepping for interviews, share this one with them.

Quick Review

Greedy sim vs full path search — when does greedy work here?

When each step has one locally-forced best move and no benefit from lookahead; otherwise BFS/DP over state space is needed.

Time complexity for simulating cell-by-cell cleaning moves?

$O(n*m)$ or $O(n*m*directions)$ — bounded by grid size times possible facing states, not exponential path enumeration.

Space complexity driver for this problem?

$O(n*m)$ for a visited/state grid tracking position plus direction/battery level, not recursion over all paths.

Trickiest bug when tracking a draining resource (battery) per move?

Off-by-one on when battery decrements — before or after the move — causes early termination or one extra step counted.

Why brute-force all paths fails on this LeetCode 3568 problem?

State space blows up combinatorially with grid size and battery levels; must collapse to deterministic/greedy transitions instead.

Interview follow-up: how would you handle a bigger grid with obstacles?

Same simulation logic still applies since transitions stay local; only bounds checking and obstacle skip logic need adding.